PYTHON PROGRAMMING

CASE STUDY ASSIGNMENT – 8

Case Study 8 – Weekly Electricity Consumption Matrix

1. Problem Statement

Monitoring electricity consumption helps households understand their energy usage and manage electricity expenses. Recording daily electricity consumption for multiple days and comparing the usage can be difficult when done manually.

The Weekly Electricity Consumption Matrix system is a Python-based program that records electricity consumption for each day of a week. It stores the daily readings in a list, calculates the total and average electricity consumption, identifies the day with the highest consumption, and displays the daily usage details. This system helps users analyze their electricity usage and identify opportunities to save energy.

2. Objectives

1. **To record electricity consumption for each day of a week.**
2. **To store daily electricity readings in a list.**
3. **To calculate the total electricity consumption for the week.**
4. **To calculate the average daily electricity consumption.**
5. **To identify the day with the highest electricity consumption.**
6. **To display the daily electricity consumption details.**
7. **To validate electricity readings and prevent negative values.**
8. **To apply Python concepts such as lists, loops, conditional statements, and arithmetic operations to analyze electricity usage.**

3. Assumptions

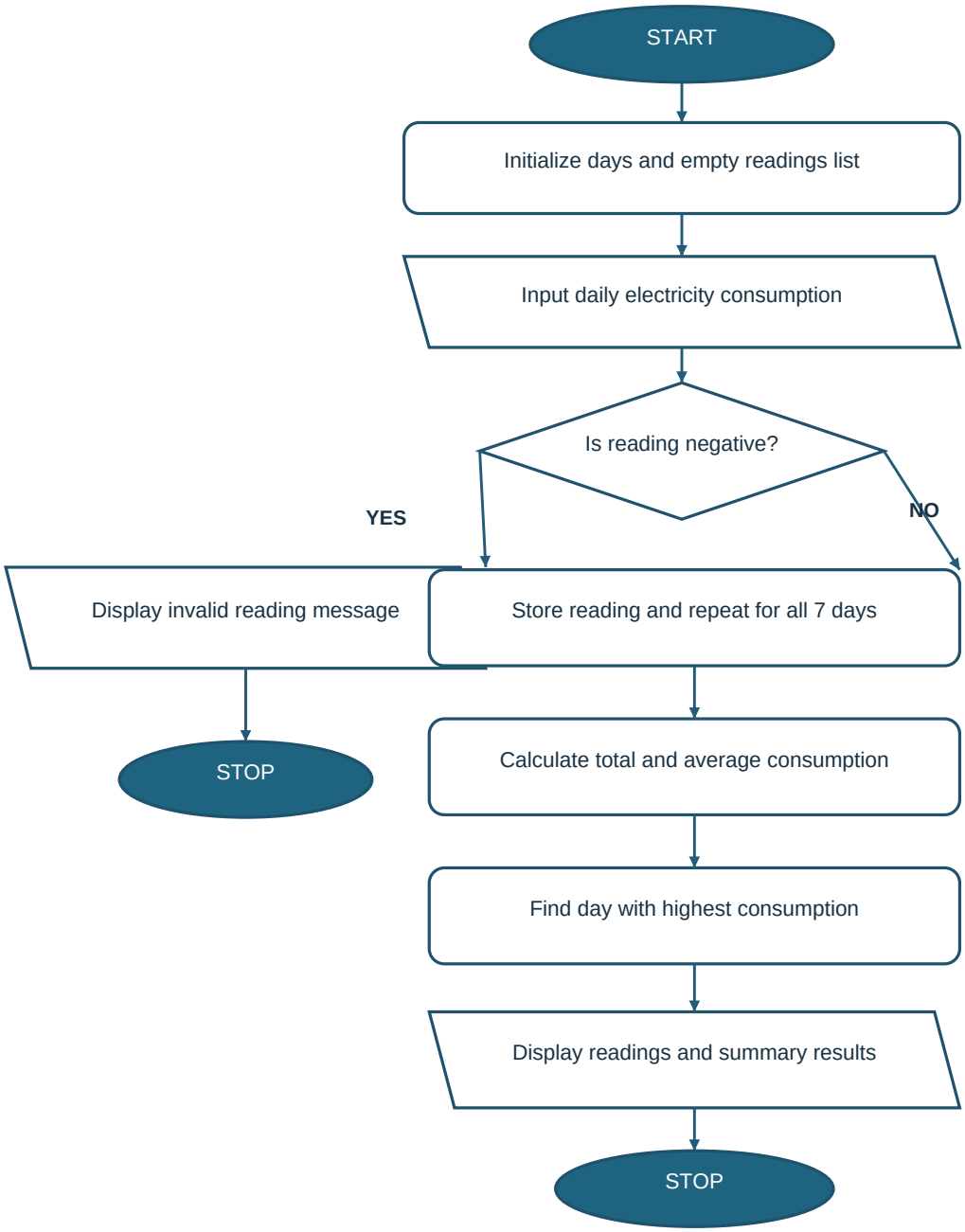
1. **The program records electricity consumption for seven days, from Monday to Sunday.**
2. **Daily electricity consumption is entered in kilowatt-hours (kWh).**
3. **All consumption values must be zero or positive numbers.**

4. **The readings are assumed to represent the electricity consumed each day.**
5. **Total weekly consumption is calculated by adding the seven daily readings.**
6. **Average daily consumption is calculated by dividing the total consumption by seven.**
7. **The day with the highest consumption is identified by comparing the daily readings.**
8. **If multiple days have the same highest consumption, the first such day is displayed.**
9. **Electricity charges, taxes, and fixed service fees are not included.**

4. Algorithm

1. **Start the program.**
2. **Initialize a list of seven days from Monday to Sunday.**
3. **Initialize an empty list to store electricity consumption readings.**
4. **Repeat for each day of the week:**
    - **Input the electricity consumption in kWh.**
    - **Check whether the reading is negative.**
    - **If the reading is negative, display an invalid reading message and stop processing.**
    - **Otherwise, store the reading in the list.**
5. **Calculate the total weekly consumption by adding all seven readings.**
6. **Calculate the average daily consumption by dividing the total by seven.**
7. **Identify the day with the highest electricity consumption.**
8. **Display the daily readings, total consumption, average consumption, and day with the highest consumption.**
9. **Stop the program.**

5. Flowchart



Standard symbols: oval = start/stop, rectangle = process, parallelogram = input/output, diamond = decision.

6. Pseudocode

```
BEGIN
SET days = Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday
SET readings = empty list
SET total = 0
SET valid = TRUE
FOR each day in days DO
  INPUT consumption
  IF consumption < 0 THEN
PRINT "Invalid electricity reading"
    SET valid = FALSE
    STOP LOOP

  ELSE
    ADD consumption to readings
    total = total + consumption
  END IF
```

END FOR

IF valid = TRUE THEN

average = total / 7

highest = maximum value in readings

highest_day = day corresponding to highest

PRINT daily readings

PRINT total

PRINT average

PRINT highest_day

PRINT highest

END IF

END

7.source code

Weekly Electricity Consumption Matrix.py

PYTHON-ACTIVITY-BASED-LEARNING > Weekly Electricity Consumption Matrix.py > ...

```python

days = ["Monday", "Tuesday", "Wednesday", "Thursday",
        "Friday", "Saturday", "Sunday"]

readings = []
total = 0
valid = True

for day in days:
    consumption = float(input("Enter electricity consumption for " + day + " (kWh): "))

    if consumption < 0:
        print("Invalid electricity reading")
        valid = False
        break

    readings.append(consumption)
    total += consumption

if valid:
    average = total / 7
    highest = max(readings)
    highest_day = days[readings.index(highest)]

    print("Daily electricity readings:", readings)
    print("Total weekly consumption:", round(total, 2), "kWh")
    print("Average daily consumption:", round(average, 2), "kWh")
    print("Day with highest consumption:", highest_day)
    print("Highest consumption:", highest, "kWh")

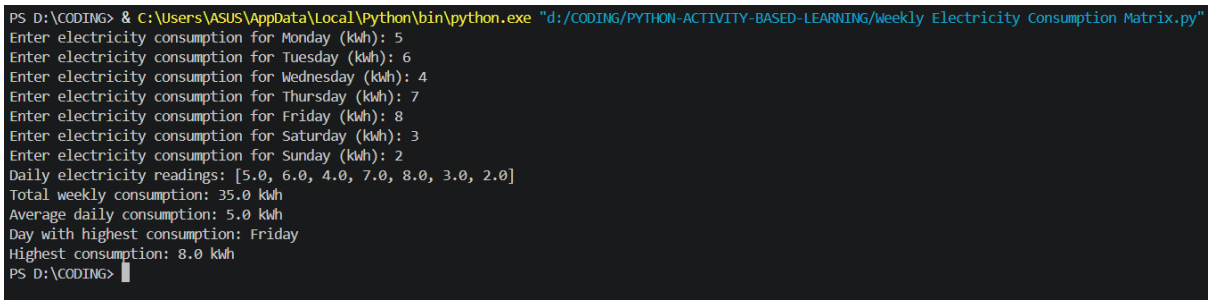
```

8. Test Cases

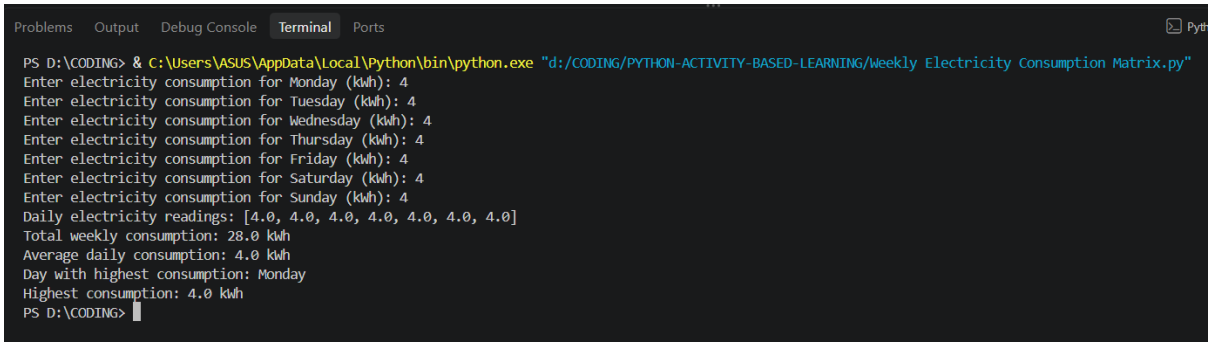
Test Case	Input	Expected Output
1. Normal consumption	Monday–Sunday: 5, 6, 4, 7, 8, 3, 2 kWh	Total: 35 kWh; Average: 5 kWh; Highest consumption: 8 kWh on Friday
2. Equal consumption	Monday–Sunday: 4, 4, 4, 4, 4, 4, 4 kWh	Total: 28 kWh; Average: 4 kWh; Highest consumption: Monday, 4 kWh
3. Zero consumption	Monday–Sunday: 0, 0, 0, 0, 0, 0, 0 kWh	Total: 0 kWh; Average: 0 kWh; Highest consumption: Monday, 0 kWh
4. Invalid reading	Monday: -2 kWh	Invalid electricity reading
5. Decimal readings	Monday–Sunday: 1.5, 2.5, 3.0, 2.0, 4.5, 1.0, 2.5 kWh	Total: 17 kWh; Average: approximately 2.43 kWh; Highest consumption: Friday, 4.5 kWh

9. Output Screenshots

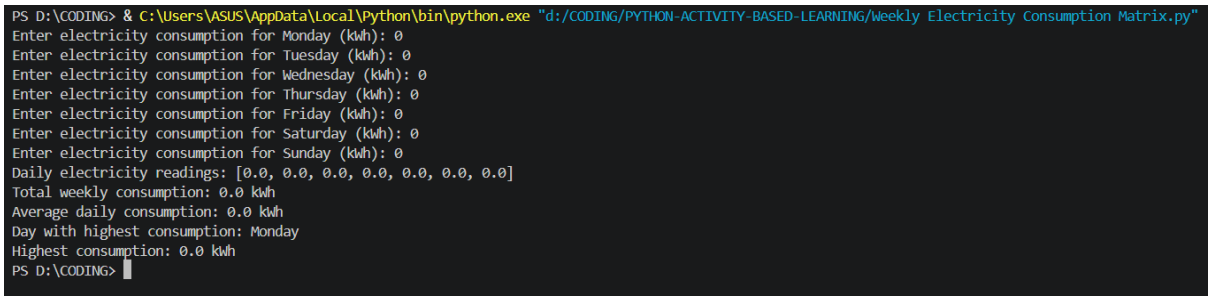
Screenshot 1: Program output showing normal weekly electricity consumption and the highest-consumption day.



Screenshot 2: Program output showing equal electricity consumption on all seven days.



Screenshot 3: Program output showing zero electricity consumption for the entire week.



10. Conclusion

The Weekly Electricity Consumption Matrix program was developed successfully using Python. It records daily electricity consumption for seven days and calculates the total and average weekly usage. The program also identifies the day with the highest electricity consumption and validates readings to prevent negative values. This project demonstrates the practical use of lists, loops, conditional statements, built-in functions, and arithmetic operations in Python. It helps users monitor their electricity usage and understand daily consumption patterns, supporting better energy management.